Week 11 Lab Handout

Resilience — When Your Partner Is Slow, Flaky, or Gone

Time: 110 minutes

Bring: your laptop, your Week 10 integration (the feature that calls your upstream partner's live API), and your `INTEGRATION_LOG.md`

Before you start: your partner will fail. Plan for it.

> Last week your feature survived a partner that was **down**: no crash, a friendly message. That is the easy failure, because a stopped server refuses the connection at once. The failures that hurt are the ones that don't announce themselves: a partner that is **slow**, one that fails **sometimes**, one that answers with the wrong data.

Every team's API will be slow or down at some point this semester: deploys, bugs, wifi. Today you stop hoping that won't happen and make your feature behave well when it does.

Part A — Reproduce the Failures First (15 min)

You can't fix what you haven't seen. First, watch your Week 10 feature fail the hard way.

Save this stand-in partner as `flaky-partner.js` and run it with `node flaky-partner.js`. It needs Node 18 or newer and Express, even if your own app uses another language: it is a separate little program, not part of your project. It is a partner **you** control, with four modes you can switch without restarting:

```
const express = require('express');
const app = express();

let mode = 'ok'; // ok | slow | flaky | error
let calls = 0;

const tasks = [
  { id: 1, title: 'Write README', dueDate: '2026-11-02', status: 'active' },
  { id: 2, title: 'Ship v1', dueDate: '2026-11-09', status: 'active' },
];

// Switch behaviour without restarting: curl http://localhost:4000/__mode/slow
app.get('/__mode/:m', (req, res) => {
  mode = req.params.m;
  calls = 0;
  res.json({ mode });
});

app.get('/tasks', async (req, res) => {
  calls += 1;
  console.log(`GET /tasks (call ${calls}, mode ${mode})`);
  if (mode === 'slow') { // answers after 10 seconds
    await new Promise(r => setTimeout(r, 10000));
  }
  if (mode === 'flaky' && calls % 3 !== 0) { // fails twice, then works
    return res.status(503).json({ message: 'Try again' });
```

```
  }
  if (mode === 'error') { // always fails
    return res.status(500).json({ message: 'Internal error' });
  }
  res.json(tasks);
});

app.listen(4000, () => console.log('Stand-in partner on http://localhost:4000'));
```

1. Point your app at it for now: `PARTNER_URL=http://localhost:4000` in your `.env`. (Your real partner goes back in at the end.)
2. **Measure normal.** With the stand-in in `ok` mode, time a call:

```
curl -s -o /dev/null -w "%{time_total} seconds\n" http://localhost:4000/tasks
```

Write the number down. You need it in Part B.

3. Switch to `slow` (`curl http://localhost:4000/__mode/slow`), reload your page, and **time how long it waits**. With the Week 10 code, your page hangs until the partner finally answers, and a user would have given up long before.
4. Switch to `flaky`. Your page fails two times out of three, even though a second attempt would have worked.

Note both results in `RESILIENCE_NOTES.md` (you'll finish it in Part F). Those are the "before" pictures.

Part B — Add a Timeout (20 min)

A call with no timeout waits as long as the partner takes. One slow partner can then tie up your whole feature. A timeout says: after this long, give up and treat it as a failure.

Add one option to your partner call:

```
const r = await fetch(`${PARTNER_URL}/tasks?status=active`, {
  headers: { 'X-API-Key': PARTNER_KEY },
  signal: AbortSignal.timeout(1500), // give up after 1.5 seconds
});
```

When the time runs out, `fetch` throws an error named `TimeoutError`. Your existing `catch` already handles failures; make the log line say what happened:

```
const reason = err.name === 'TimeoutError'
  ? 'timeout' : (err.cause?.code || err.message);
console.error('partner call failed:', reason);
```

(In another language, the idea is identical: set the timeout option on your HTTP client, for example `timeout` in Guzzle or `timeout=` in Python's `requests`.)

Choosing the number. A timeout needs to be longer than a normal response (use the number you measured in Part A, then leave room for a bad moment) and shorter than your user's patience. 1500 ms is a reasonable starting point for a call that normally takes well under half a second. Write down your number and **why** in `RESILIENCE_NOTES.md`.

Prove it. With the stand-in in `slow` mode, reload your page. You should now get your friendly failure message after about 1.5 seconds instead of waiting 10.

Part C — Add Retry Logic (30 min)

Some failures are momentary: a deploy restarting, a dropped connection, a busy moment. A second attempt a moment later often works. But retrying carelessly causes its own damage, so the rules matter more than the code:

Retry?	Situation
Yes	The call timed out, or the connection failed
Yes	The partner returned 502, 503, or 504 (or another 5xx)
Yes, but wait	The partner returned 429. Wait for their `Retry-After` time
No	The partner returned 400, 401, 403, or 404. Your request is wrong, and asking again changes nothing
No	The request **changes data** (POST, PUT, PATCH, DELETE). A retry may do it twice. This is the idempotency idea from Week 3

Two more rules: **never retry forever** (this feature allows at most 2 retries), and **wait between attempts** (short, and longer each time).

Put this in its own file, `resilientFetch.js`, and use it instead of calling `fetch` directly:

```
const RETRY_STATUSES = new Set([429, 500, 502, 503, 504]);
const sleep = ms => new Promise(resolve => setTimeout(resolve, ms));

async function fetchWithRetry(url, options = {}, config = {}) {
  const { timeoutMs = 1500, retries = 2, delayMs = 200 } = config;
  const method = (options.method || 'GET').toUpperCase();
  const maxRetries = method === 'GET' ? retries : 0; // never retry writes

  for (let attempt = 0; ; attempt++) {
    let wait = delayMs * 2 ** attempt; // 200 ms, then 400 ms
    try {
      const res = await fetch(url, {
        ...options,
        signal: AbortSignal.timeout(timeoutMs),
      });
      if (!RETRY_STATUSES.has(res.status) || attempt >= maxRetries) return res;
      const retryAfter = Number(res.headers.get('Retry-After'));
      if (res.status === 429 && retryAfter > 0) {
        wait = Math.min(retryAfter * 1000, 5000); // wait as asked, up to 5 s
      }
    } catch (err) {
      if (attempt >= maxRetries) throw err; // out of retries
    }
    console.warn(`attempt ${attempt + 1} failed, retrying in ${wait} ms`);
    await sleep(wait);
  }
}
```

```
module.exports = { fetchWithRetry };
```

Read it before you paste it. Where is the timeout? Where does it stop retrying? Why does a `404` come straight back instead of looping?

Then your route calls it:

```
const { fetchWithRetry } = require('./resilientFetch');

const r = await fetchWithRetry(
  `${PARTNER_URL}/tasks?status=active`,
  { headers: { 'X-API-Key': PARTNER_KEY } },
  { timeoutMs: 1500, retries: 2, delayMs: 200 }
);
if (!r.ok) throw new Error(`partner returned ${r.status}`);
```

Do the arithmetic. In the worst case your user waits for every attempt to time out:

```
(retries + 1) x timeout + delays = 3 x 1500 + 200 + 400 = 5100 ms
```

That is more than five seconds before your page gives up. Is that acceptable for your feature? If not, lower the timeout or the retries. Write the worst-case number in your notes.

Prove it.

1. Stand-in in `flaky` mode: your page should now **succeed**, and your terminal should show two "retrying" lines for each page load. The user never saw a failure.
2. Stand-in in `error` mode (always 500): your terminal should show exactly **two** retries and then a failure. If it keeps going, you have an infinite loop. Stop and fix it.

Part D — Add a Graceful Fallback (25 min)

When every attempt fails, you still choose what the user sees. From best to worst:

1. **Live data** (everything worked)
2. **Last known good data**, clearly labeled as saved, with the time it was saved
3. **A friendly "data unavailable" message** (you have nothing saved)
4. A blank page, a spinner that never ends, or a crash. This is what you are avoiding.

Keep the last good response in memory and return it, flagged, when the partner can't be reached. Update your `/upcoming` route:

```
let lastGood = null; // { data, asOf }, kept in memory

app.get('/upcoming', async (req, res) => {
  try {
    const r = await fetchWithRetry(
      `${PARTNER_URL}/tasks?status=active`,
      { headers: { 'X-API-Key': PARTNER_KEY } },
      { timeoutMs: 1500, retries: 2, delayMs: 200 }
    );
    if (!r.ok) throw new Error(`partner returned ${r.status}`);
```

```
    const tasks = await r.json();
    const data = tasks.map(t => ({ title: t.title, dueDate: t.dueDate }));
    lastGood = { data, asOf: new Date().toISOString() };
    return res.json({ data, stale: false, asOf: lastGood.asOf });
  } catch (err) {
    const reason = err.name === 'TimeoutError'
      ? 'timeout' : (err.cause?.code || err.message);
    console.error('partner call failed:', reason);
    if (lastGood) {
      return res.json({ data: lastGood.data, stale: true, asOf: lastGood.asOf });
    }
    return res.status(502).json({ message: 'Partner data unavailable' });
  }
});
```

Notice the response is now an object with `data`, `stale`, and `asOf`, not a bare list. Your frontend needs a matching change, and **the fallback has to be visible**:

```
const res = await fetch('/upcoming');
if (!res.ok) {
  showMessage('Partner data is unavailable right now. Try again in a minute.');
  return;
}
const { data, stale, asOf } = await res.json();
render(data);
if (stale) {
  const time = new Date(asOf).toLocaleTimeString();
  showBanner(`Showing saved data from ${time}. The partner is not responding.`);
}
```

If users can't tell the data is old, a fallback becomes a lie. The banner is part of the feature.

Two things to know. The saved copy lives in memory, so it disappears when you restart your server (fine for this course; production systems use a cache or a database). And the log still records every failure. If the real cause is a `401` from a wrong key, a saved copy hides it from users, so your log is how you find out.

Prove it.

1. Stand-in in `ok` mode: load the page. Live data, no banner.
2. Switch to `slow`, then reload. After about five seconds you see the **same data with the saved-data banner**, not an error.
3. Restart your server (this clears the saved copy), keep the stand-in in `slow` mode, and reload. Now you see the friendly "unavailable" message. No crash.

Part E — Prove It Against Your Real Partner (10 min)

Point `PARTNER_URL` back at your real upstream partner and run the same drill with them. Agree a short window with your partner first. They are about to turn their server off and on for you, and you for them.

1. **Normal:** the page shows live data.
2. **Partner stopped:** ask your partner to stop their server. Within the worst-case time you calculated, the page shows saved data with the banner.
3. **Partner back:** ask them to restart. Reload: live data again, banner gone, and you changed no code.

4. **Be a good consumer in return.** Watch their terminal if they show it to you. Your retries should appear as at most three hits for each page load, never a flood. Aggressive retries make a struggling partner worse, which is why you wait between attempts and honor `Retry-After`.

Then look at your own terminal: how many calls is your downstream partner making to **you**? If you see bursts of three, those are their retries. Check they are not hitting your rate limit from Week 9.

Part F — Write It Up and Push (10 min)

Finish `RESILIENCE_NOTES.md` in your repo root. It needs:

1. **A table of what you tested:** one row per failure (partner stopped, slow, flaky, and one more of your choice), with what the user saw and what your log said.
2. **Your numbers, and why:** timeout, retries, delay, and the worst-case wait. Say what you measured to choose them.
3. **What would a real user experience without this code?** Write 4–6 sentences, specific to your feature. Name the screen, the wait, and what the user does next.
4. **What you did not build.** One sentence: a *circuit breaker* is a switch that stops calling a partner that has been failing for a while, and tests it occasionally. Yours still tries every request three times. You are not implementing it, but you should be able to say what it would change.

Then commit and push:

```
git add . RESILIENCE_NOTES.md resilientFetch.js
git commit -m "Week 11: timeout, retry, and graceful fallback for partner calls"
git push
```

Before you push, run `git status` and confirm `.env` is **not** listed.

This week's deliverable checklist

- The partner call has a timeout, and a slow partner is cut off at that time
- Retries are capped at 2, wait between attempts, and never happen for 4xx errors or writes
- With the partner down or slow, the page shows saved data with a visible banner, or a friendly message if nothing is saved
- A demo or screenshot of the fallback state with the partner intentionally stopped
- `RESILIENCE_NOTES.md`: test table, your numbers and why, user impact, and the circuit breaker sentence
- The log says why each failure happened, and never contains a key
- The real partner drill is done, and `PARTNER_URL` points back at them
- All code committed and pushed, `.env` not committed

Rubric (20 pts)

Criterion	Points
Timeout implemented correctly	5
Retry logic implemented sensibly (not infinite, not on unsafe writes)	6
Graceful fallback demonstrated working	6
Write-up shows real understanding of user impact	3

Common problems and quick fixes

Problem	Fix
The timeout never fires	Check you pass `signal: AbortSignal.timeout(...)` inside the `fetch` options. You need Node 18 or newer. A timeout set on something else does not cancel the request
It retries forever	The loop has no exit. `attempt >= maxRetries` must return or throw. Test with the stand-in in `error` mode and count the terminal lines
It retries a 404 or 401	Only the statuses in `RETRY_STATUSES` should loop. A wrong key or path fails identically every time. Fix the request
The page waits far too long	Work out the worst case: (retries + 1) x timeout + delays. Lower the timeout or the retries
You never see the saved-data banner	Check the frontend reads `stale` and `asOf` from the new response object, and that you loaded live data at least once before stopping the partner
Saved data shows after you restart your server	It shouldn't. The copy lives in memory. If it survives, you are caching somewhere else, so check the code
The retry made things worse for the partner	Check the delay grows (200 ms, then 400 ms) and honors `Retry-After`. Never retry in a tight loop

Next week: Monitoring. Today your feature coped with failures it could see in the terminal. Next you'll add logs and simple counts, so you can tell how often it is happening without watching.